Kadi-APY for everyday lab work

Online workshop, participant guide. Please complete the checklist on this page before the workshop. It takes about 20 minutes.

DATE & TIME	MEETING LINK
Friday, 13 November 2026, 14:00-16:00 (CET)	**[FILL IN]**

Checklist before the workshop

- [ ] **1. Log in to Kadi4Mat**

  Open **kadi4mat.iam.kit.edu** and log in. Kadi4Mat is the research data repository used in the workshop.

- [ ] **2. Create a personal access token**

  The token allows kadi-apy to access Kadi4Mat on your behalf. In Kadi4Mat:

  1. Click your name (top right), then Settings > Access tokens.
  2. Name it `workshop`, set an expiry date a few days after the workshop, and create it.
  3. Copy the token right away, it is only shown once.

- [ ] **3. Check your Google account**

  The workshop uses Google Colab, which runs Python in the web browser. Open **colab.research.google.com** and sign in with a Google account.

- [ ] **4. Open the workshop notebook and save a copy**

  Open **[FILL IN short link]**, then File > Save a copy in Drive. Work in your copy.

- [ ] **5. Store your token as a Colab secret**

  In the notebook, click the key icon in the left sidebar, then "Add new secret". Name: `KADI_PAT`, value: your token. Switch on "Notebook access".

- [ ] **6. Run the setup cells (section 0)**

  Run the cells one after the other with `Shift` + `Enter`. In cell 0.4, replace `"xy"` with your initials. The setup is complete when the output shows "Setup complete", "Connected to ..." and "Prefix set to ...".

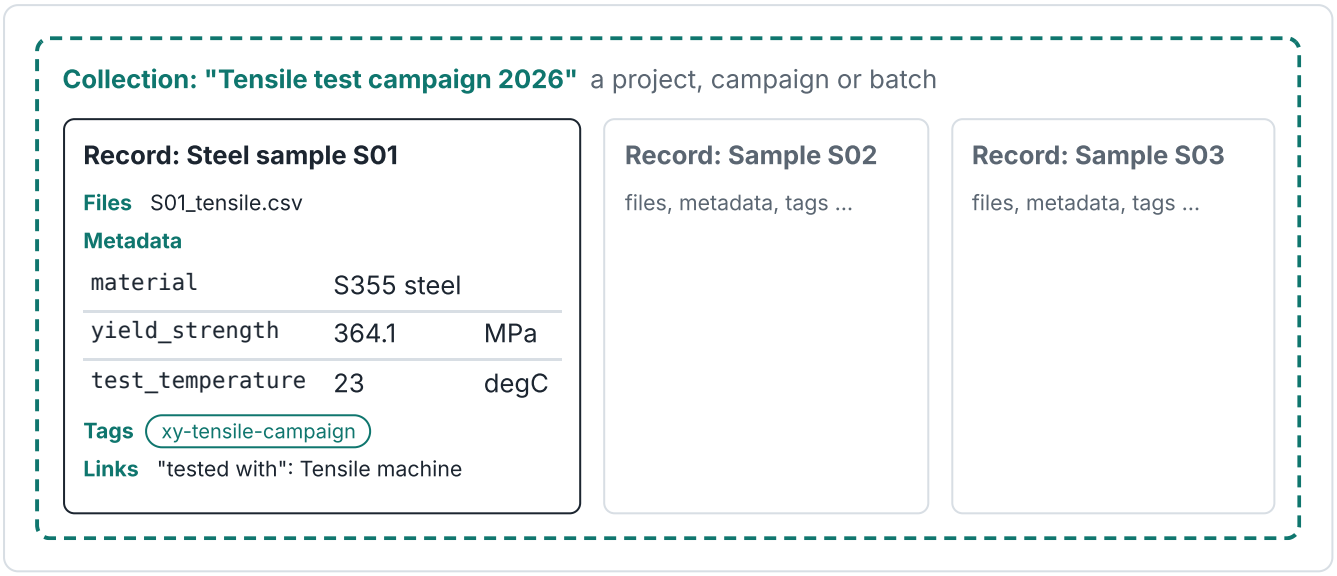
- [ ] **7. Prepare your screen**

  Test your microphone and the meeting chat. If possible, use two screens or place the meeting and the notebook side by side. Keep Kadi4Mat open in a second browser tab.

> **Problems with the setup?** Please reply to the invitation e-mail with a screenshot of the error before the workshop, or join the meeting 10 minutes early.

Kadi4Mat basics

Kadi4Mat stores research data together with structured, searchable metadata. The main terms used in the workshop:



TERM	MEANING
Record	One documented object, e.g. a sample, a measurement or a simulation run. Contains files and metadata.
Metadata	Structured information about a record: key (name), type (text, number, ...), value and unit. Metadata can be searched.
Collection	A group of records, e.g. a project or a test campaign.
Tag	A keyword for finding records.
Identifier	A short name for a record, e.g. `jd-sample-s01`. It must be unique in Kadi4Mat, so in the workshop it starts with your initials.

What is kadi-apy?

kadi-apy is a Python library and command line tool for the Kadi4Mat API. Most actions of the web interface can be scripted with it, which is useful when the same steps are repeated for many files.

Website
Manual work, suitable for a few records.

Command line
`kadi-apy records ...`
One command per action.

Python
`from kadi_apy import ...`
Loops, calculations, reading files.

> **Typical uses:** uploading a folder of instrument files; extracting results (e.g. yield strength) into metadata; consistent names and tags within a group; downloading data for a report or paper.

Working in Google Colab

TASK	HOW
Run a cell	`Shift` + `Enter`, or the run button on the left of the cell.
Order	From top to bottom. Later cells use results of earlier ones.
Structure	Each part: example, reference with documentation links, exercise.
Exercises	Change only lines marked `TODO`. Solution: "Show solution".
Lines starting with `!`	Run as terminal commands, not as Python.
Reset	Runtime > Restart session, then run the cells again from the top.
Errors	The last line of the error message usually names the problem.

Using the command line (CLI)

All kadi-apy commands have the same structure:

`kadi-apy`	`records`	`add-metadatum`	`-R jd-sample-s01 -m length -v 80 -t float -u mm`
the program	resource type: records, collections, ...	the action	options, each as **-letter value**

Where to run commands

Colab: in a code cell, with `!` in front:

```
!kadi-apy records --help
```

Own computer: in a terminal (Windows: *Anaconda Prompt* or *Command Prompt*; macOS: *Terminal*), without `!`. One-time setup:

```
pip install kadi-apy
kadi-apy config create    # new config
kadi-apy config set-host  # Kadi4Mat URL
kadi-apy config set-pat   # your token
```

Notes

Values with spaces need quotes: `-t "Steel sample S01"`.
Options are case-sensitive: `-R` and `-r` are different.
`--help` after any command lists its options.

Options used in the workshop

OPTION	MEANING
`-i`	identifier of a **new** record (`create`)
`-R`	identifier of an **existing** record
`-t`	title (`create`), tag (`add-tag`) or type (`add-metadatum`)
`-n`	file or folder to upload
`-m -v`	metadata name and value
`-u`	unit, e.g. `mm` , `MPa`
`-f`	overwrite existing files or metadata
`--help`	show all options of a command

During the workshop
Please write in the chat when you have finished an exercise, and post questions or error messages there at any time.

After the workshop
Documentation: kadi-apy.readthedocs.io
Material: github.com/msishuvo/thematic-workshop-kadiAPY
For unpublished lab data, check your institute's data policy before using Colab.

Command reference

Full documentation: **kadi-apy.readthedocs.io**, pages Usage > Library (Python) and Usage > CLI. In Colab: `help(Record.upload_file)` or `!kadi-apy records add-files --help`.

Command line (in Colab, put `!` in front)

```
# create a record
kadi-apy records create -i jd-sample-s01 -t "Sample S01"
# upload a file (or a whole folder)
kadi-apy records add-files -R jd-sample-s01 -n data/tensile_tests/S01_tensile.csv
# add metadata with type and unit
kadi-apy records add-metadatum -R jd-sample-s01 -m specimen_length -v 80 -t float -u mm
# add a tag
kadi-apy records add-tag -R jd-sample-s01 -t jd-tensile-campaign
# show metadata (-m) and file list (-l)
kadi-apy records show-info -R jd-sample-s01 -m -l
# find records by tag / list all commands
kadi-apy records get-records -t jd-tensile-campaign
kadi-apy records --help
```

Python

```
from kadi_apy import KadiManager
manager = KadiManager()                        # connection, reads the config file

# get the record, or create it if it does not exist yet
record = manager.record(identifier="jd-sample-s01", title="Sample S01", create=True)

# metadata: a list of dictionaries (key, type, value, unit)
record.add_metadata([{"key": "specimen_length", "type": "float",
                      "value": 80.0, "unit": "mm"}], force=True)

record.upload_file("data/tensile_tests/S01_tensile.csv", force=True)   # upload a file
record.add_tag("jd-tensile-campaign")                    # add a tag

# put the record into a collection
collection = manager.collection(identifier="jd-campaign", title="Campaign", create=True)
record.add_collection_link(collection.id)
```

Common errors

MESSAGE	CAUSE AND SOLUTION
NameError: name 'manager' is not defined	An earlier cell was not run, e.g. after a restart. Run section 0 again.
Could not read the secret KADI_PAT	Secret name misspelled or "Notebook access" switched off.
401 / "invalid access token"	Token incomplete or expired. Create a new token and update the secret.
identifier ... already in use	The identifier belongs to someone else. Check your prefix.
IndentationError	Wrong number of spaces at the start of a line. Compare with the solution.
SyntaxError	Usually a missing quote, bracket or comma. Compare with the solution.